Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8

по курсу «Методы оптимизации»

«Метод Пауэлла»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Необходимо реализовать метод Пауэлла

2 Результаты

Исходный код программы представлен в листинге 1.

```
using Plots
using LinearAlgebra

rosenbrock(x) = (1.0 - x[1])^2 + 100.0 * (x[2] - x[1]^2)^2
rastrigin(x) = 20.0 + (x[1]^2 - 10.0 * cos(2    * x[1])) + (x[2]^2 -
    10.0 * cos(2    * x[2]))
schwefel(x) = 418.9829 * 2.0 - x[1] * sin(sqrt(abs(x[1]))) - x[2] *
    sin(sqrt(abs(x[2])))

function find_interval(f, x, d; h=0.1, factor=2.0, max_iter=20)
    a, fa = 0.0, f(x)
    b, fb = h, f(x + h * d)
    if fb > fa
        h = -h
        b, fb = h, f(x + h * d)
        fb > fa && return (-abs(h), abs(h))
    end
    c = b + h
    for _ in 1:max_iter
        fc = f(x + c * d)
        fc > fb && return (min(a, c), max(a, c))
        a, fa, b, fb = b, fb, c, fc
        h *= factor
        c = b + h
    end
    return (a, c)
end

function line_search(f, x, d; tol=1e-4)
    a, b = find_interval(f, x, d)
    a == b && return x
       = (sqrt(5.0) - 1.0) / 2.0
    c = b -    * (b - a)
    d_pt = a +    * (b - a)
    fc, fd = f(x + c * d), f(x + d_pt * d)
```

```
    while abs(b - a) > tol
        if fc < fd
            b, d_pt, fd = d_pt, c, fc
            c = b -     * (b - a)
            fc = f(x + c * d)
        else
            a, c, fc = c, d_pt, fd
            d_pt = a +     * (b - a)
            fd = f(x + d_pt * d)
        end
    end
    return x + 0.5 * (a + b) * d
end

function powell(f, x0; max_iter=50, tol=1e-4)
    n = length(x0)
    x = copy(x0)
    dirs = [[Float64(i == j) for j in 1:n] for i in 1:n]
    trajectory = [copy(x)]
    cycle_dirs = [deepcopy(dirs)]
    iters = 0
    for k in 1:max_iter
        iters = k
        x_start = copy(x)
        max_decrease, idx_replace = 0.0, 1
        for i in 1:n
            f_old = f(x)
            x = line_search(f, x, dirs[i])
            push!(trajectory, copy(x))
            decrease = f_old - f(x)
            if decrease > max_decrease
                max_decrease, idx_replace = decrease, i
            end
        end
        d_new = x - x_start
        if abs(det(hcat(dirs...))) >= 1e-6 && norm(d_new) > 1e-8
            dirs[idx_replace] = d_new ./ norm(d_new)
        end
        push!(cycle_dirs, deepcopy(dirs))
        norm(x - x_start) < tol && break
    end
    return x, trajectory, iters, cycle_dirs
end

```

```
function plot_trajectory(f, trajectory, cycle_dirs, xlim, ylim, title, filename; arrow_scale=0.15, step=1)
    n = length(trajectory[1])
    xs = range(xlim[1], xlim[2], length=100)
    ys = range(ylim[1], ylim[2], length=100)
    Z = [f([x, y]) for y in ys, x in xs]
    plt = contour(xs, ys, Z, levels=20)
    tr = trajectory
    plot!(plt, [p[1] for p in tr], [p[2] for p in tr], color=:red, linewidth=2, label="")
    scatter!(plt, [tr[1][1]], [tr[1][2]], color=:green, markersize=8, label="start")
    scatter!(plt, [tr[end][1]], [tr[end][2]], color=:blue, markersize=8, label="end")
    max_idx = length(tr) - 1
    for i in 1:step:max_idx
        cycle_idx = min(div(i - 1, n) + 1, length(cycle_dirs) - 1)
        dirs = cycle_dirs[cycle_idx]
        px, py = tr[i][1], tr[i][2]
        scale = arrow_scale * (xlim[2] - xlim[1])
        for d in dirs
            norm(d) < 1e-8 && continue
            end_x = px + d[1] * scale
            end_y = py + d[2] * scale
            plot!(plt, [px, end_x], [py, end_y], arrow=:closed, color=:blue, linewidth=1, label="")
        end
    end
    title!(plt, title)
    xlabel!(plt, "x_1")
    ylabel!(plt, "x_2")
    savefig(plt, filename)
end

x0_rastrigin = [0.3, 0.3]
x0_schwefel = [350.0, 350.0]
x0_rosenbrock = [-1.5, 2.0]

println("                                        ")
x_opt, traj, iters, cycle_dirs = powell(rastrigin, x0_rastrigin)
println("  x* = ", x_opt, "\n  f(x*) = ", rastrigin(x_opt), "\n                    : ", iters)
plot_trajectory(rastrigin, traj, cycle_dirs, (-5, 5), (-5, 5), "                    ", "rastrigin.png", step=1)

```

```
println("                                    ")
x_opt, traj, iters, cycle_dirs = powell(schwefel, x0_schwefel)
println("  x* = ", x_opt, "\n  f(x*) = ", schwefel(x_opt), "\n
                    : ", iters)
plot_trajectory(schwefel, traj, cycle_dirs, (200, 450), (200, 450),
    "                    ", "schwefel.png", step=1)

println("                                        ")
x_opt, traj, iters, cycle_dirs = powell(rosenbrock, x0_rosenbrock)
println("  x* = ", x_opt, "\n  f(x*) = ", rosenbrock(x_opt), "\n
                    : ", iters)
plot_trajectory(rosenbrock, traj, cycle_dirs, (-2, 2), (-1, 5), "
                    ", "rosenbrock.png", step=4)
```

Результат представлен на рисунках 1– 3.

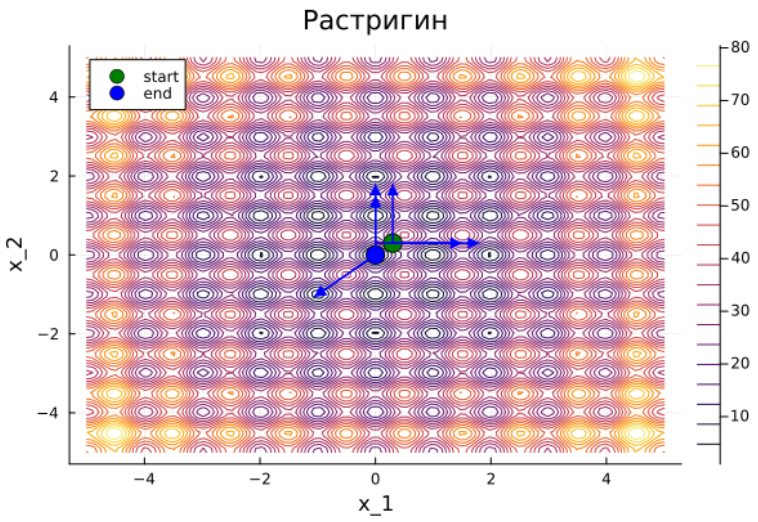

Рис. 1 — Результат

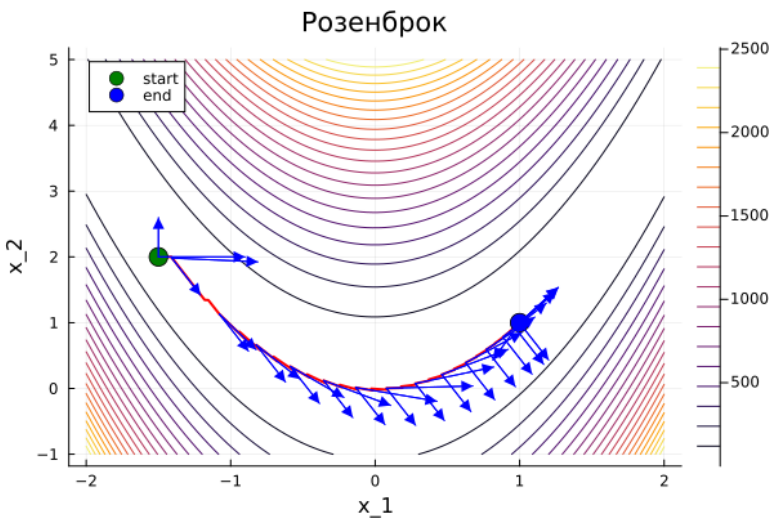

Рис. 2 — Результат

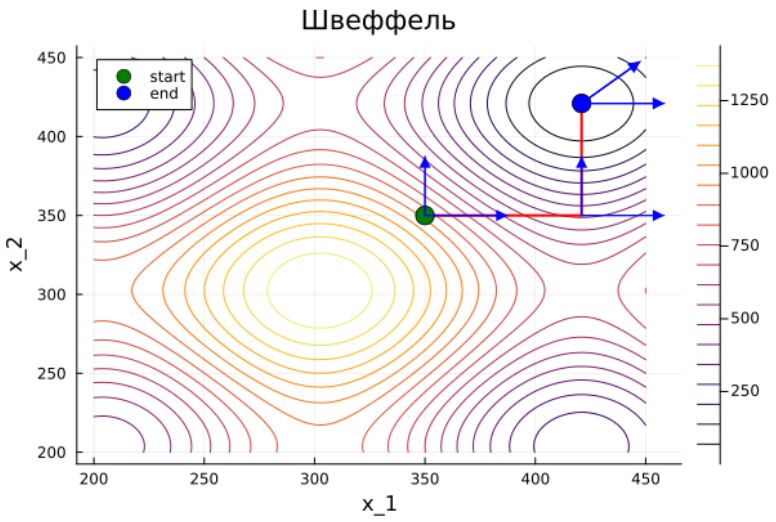

Рис. 3 — Результат